

Exercícios — Aula 03

Curso de Formação de Programação Aplicada · Módulo 1 · FPA Labs

Os exercícios 3 em diante precisam de **internet**. Se a sua conexão cair, faça os exercícios 1, 2 e 6 (que funcionam offline) e volte nos outros depois.

APIs públicas que vamos usar (todas gratuitas e sem cadastro):

API	Endereço	O que devolve
ViaCEP	<code>https://viacep.com.br/ws/{cep}/json/</code>	endereço a partir do CEP
Brasil API — DDD	<code>https://brasilapi.com.br/api/ddd/v1/{ddd}</code>	idades de um DDD
Brasil API — Feriados	<code>https://brasilapi.com.br/api/feriados/v1/{ano}</code>	feriados nacionais
PokéAPI	<code>https://pokeapi.co/api/v2/pokemon/{nome}</code>	dados de um Pokémon

Nível 1 — obrigatório

Exercício 1 — Prevendo a ordem (offline)

Antes de rodar, **escreva num papel** em que ordem as linhas vão aparecer:

```
console.log("A");

setTimeout(() => console.log("B"), 1000);
setTimeout(() => console.log("C"), 0);

Promise.resolve().then(() => console.log("D"));

console.log("E");
```

Depois rode e compare. Se errou, escreva num comentário no arquivo **por que** a ordem é aquela.

Exercício 2 — Semáforo (offline)

Em `02-semaforo.js`, crie uma função `esperar(ms)` que devolve uma Promise e use-a para simular um semáforo em loop infinito:

● VERDE (3s)
● AMARELO (1s)
● VERMELHO (4s)
● VERDE (3s)
...

Dica: `esperar` é `new Promise((resolve) => setTimeout(resolve, ms))`. Use um `while (true)` dentro de uma função `async`. Encerre com `Ctrl+C`.

Exercício 3 — Buscando um CEP

Em `03-cep.js`, receba um CEP pelo terminal e imprima o endereço formatado:

```
node 03-cep.js 20040030
```

Rua do Ouvidor, Centro
Rio de Janeiro - RJ

Requisitos:

- Use `async/await` e `try/catch`.
- Trate o caso do CEP não existir (o ViaCEP devolve `{ "erro": true }` com status 200 — pegadinha!).
- Se nenhum CEP for informado, mostre como usar o comando.

Nível 2 — esperado

Exercício 4 — Vários CEPs em paralelo

Em `04-varios-ceps.js`, busque estes 5 CEPs e monte uma tabela com cidade e UF de cada:

```
01001000 20040030 30130010 80010000 90010000
```

Requisitos:

- Faça as buscas em **paralelo** com `Promise.all`.
- Meça o tempo com `performance.now()` e imprima no final.
- Depois, faça a mesma coisa em sequência (com `for...of`) e **compare os dois tempos** num comentário no arquivo.

Exercício 5 — Feriados do ano

Em `05-feriados.js`, use a Brasil API para:

1. Buscar os feriados nacionais de um ano informado pelo terminal.
2. Imprimir cada feriado com data formatada em `DD/MM`.
3. Mostrar **quantos** feriados caem em dia útil (segunda a sexta).
4. Mostrar qual é o **próximo** feriado a partir de hoje (se o ano for o atual).

Dicas: `new Date("2026-01-01")` cria uma data; `.getDay()` devolve 0 (domingo) a 6 (sábado).

Desafio — opcional

Exercício 6 — Cache em arquivo (offline + online)

Buscar a mesma informação na internet toda vez é lento e desnecessário. Faça um buscador de CEP com **cache local**:

1. Ao buscar um CEP, primeiro procure no arquivo `cache.json`.
2. Se estiver lá → use e imprima `(do cache)`.
3. Se não estiver → busque na API, **salve no cache** e imprima `(da API)`.

```
node 06-cache.js 01001000 # (da API) - demorou 300ms
node 06-cache.js 01001000 # (do cache) - instantâneo
```

Requisitos:

- Use `readFile` / `writeFile` de `node:fs/promises`.
- Trate o caso de o `cache.json` ainda não existir (erro com `code === "ENOENT"`).
- Não mute o objeto de cache: use spread para criar a versão nova.

Exercício 7 — Comparador de Pokémon

Em `07-pokemon.js`, receba dois nomes de Pokémon e compare-os:

```
node 07-pokemon.js pikachu charmander
```

Mostre para cada um: nome, altura, peso e tipos. Depois diga qual é o mais pesado.

Requisitos:

- Busque os dois em **paralelo**.
- Se um dos nomes não existir, a API devolve status 404 — trate isso com uma mensagem amigável (lembre: `fetch` **não** lança erro sozinho em 404).

Checklist de entrega

- ☐ Nenhum `console.log` meu imprimiu `Promise { <pending> }`
- ☐ Toda chamada de rede está dentro de um `try/catch`
- ☐ Verifiquei `resposta.ok` antes de usar os dados
- ☐ Onde as chamadas eram independentes, usei `Promise.all`
- ☐ Não usei `forEach` com `async` dentro